Integration Guide

Step-by-step instructions for integrating the SDK into existing Solana projects.

Table of Contents

1. Prerequisites
2. Installation
3. Quick Start
4. Basic Integration
5. Advanced Features
6. Testing
7. Deployment

Prerequisites

Before integrating the SDK, ensure you have:

- **Node.js** 18+ installed
- **Solana CLI** 1.18+ installed
- **Anchor** 0.30+ installed
- **Rust** 1.70+ installed
- A Solana wallet with SOL for testing

Required Tools

```
# Install Solana CLI
sh -c "$(curl -sSfL https://release.solana.com/v1.18.1/install)"

# Install Anchor
cargo install --git https://github.com/coral-xyz/anchor avm --locked
avm install 0.30.1
avm use 0.30.1

# Verify installations
solana --version
anchor --version
```

Installation

1. Add SDK to Your Project

```
npm install @gitdigital/solana-kyc-sdk
# or
yarn add @gitdigital/solana-kyc-sdk
```

2. Install Peer Dependencies

```
npm install @solana/web3.js @coral-xyz/anchor @solana/spl-token
```

3. Configure Environment

Create a `.env` file:

```
# RPC Configuration
RPC_URL=https://api.devnet.solana.com
SOLANA_CLUSTER=devnet

# Program ID (use devnet ID for testing)
PROGRAM_ID=KYCCo7vM2uLkGzqH6XqKJp1TJK5JjK9jW8vY9xQz1P2

# Optional: Admin keypair for privileged operations
ADMIN_KEY=/path/to/admin/keypair.json

# Optional: API keys for external services
CHAINALYSIS_API_KEY=your_key
TRM_LABS_API_KEY=your_key
```

Quick Start

1. Initialize the SDK

```
import { createComplianceSDK, KycLevel } from '@gitdigital/solana-kyc-
sdk';
import { Connection, PublicKey } from '@solana/web3.js';

const sdk = createComplianceSDK({
  rpcUrl: process.env.RPC_URL || 'https://api.devnet.solana.com',
  programId: new PublicKey(process.env.PROGRAM_ID!),
  commitment: 'confirmed',
});
```

2. Register a User

```
const expiryTimestamp = Math.floor(Date.now() / 1000) + 365 * 24 * 60
* 60;

await sdk.registerUser({
  wallet: userWallet,
  kycLevel: KycLevel.Standard,
  countryCode: 'US',
  expiryTimestamp,
});
```

3. Check Compliance Status

```
const status = await sdk.checkComplianceStatus(walletAddress);

if (status.status === 'verified') {
  // Allow operation
} else {
  // Handle accordingly
}
```

Basic Integration

Step 1: Set Up Your Project

```
// src/kyc.ts
import { createComplianceSDK, KycLevel } from '@gitdigital/solana-kyc-
sdk';
import { PublicKey } from '@solana/web3.js';

const PROGRAM_ID = new
PublicKey('KYCCo7vM2uLkGzqH6XqKJp1TJK5JjK9jW8vY9xQz1P2');

export const kycSDK = createComplianceSDK({
  rpcUrl: process.env.RPC_URL!,
  programId: PROGRAM_ID,
  commitment: 'confirmed',
});

export { KycLevel };
```

Step 2: Integrate with Your Application

For a Web Application

```
// In your React/Vue/Svelte component
import { kycSDK, KycLevel } from './kyc';

async function handleUserRegistration(userWallet: PublicKey) {
  try {
    await kycSDK.registerUser({
      wallet: userWallet,
      kycLevel: KycLevel.Standard,
      countryCode: 'US',
      expiryTimestamp: Math.floor(Date.now() / 1000) + 365 * 24 * 60 *
60,
    });

    alert('Registration successful!');
  } catch (error) {
    console.error('Registration failed:', error);
    alert('Registration failed. Please try again.');
  }
}
```

For a Backend Service

```
// In your backend handler
import { kycSDK, KycLevel } from './kyc';

app.post('/api/register', async (req, res) => {
  const { walletAddress, countryCode, kycLevel } = req.body;

  try {
    const signature = await kycSDK.registerUser({
      wallet: new PublicKey(walletAddress),
      kycLevel: kycLevel as KycLevel,
      countryCode,
      expiryTimestamp: Math.floor(Date.now() / 1000) + 365 * 24 * 60 *
60,
    });

    res.json({ success: true, signature });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

Step 3: Enforce KYC on Transfers

```
async function processTransfer(sender: PublicKey, recipient:
PublicKey, amount: BN) {
  // Check compliance first
  const result = await kycSDK.approveTransfer(sender, recipient,
amount);

  if (!result.approved) {
    throw new Error(`Transfer not allowed: ${result.reason}`);
  }

  // Proceed with your transfer logic
  // ...
}
```

Advanced Features

Creating a Compliant Token

```
// Create a new SPL Token with KYC enforcement
const { mint, signature } = await kycSDK.createCompliantMint(
  authority,  // Your admin wallet
  6           // Decimals (e.g., 6 for USDC-like tokens)
);

// The mint is now configured to enforce KYC on all transfers
```

Managing High-Risk Accounts

```
// Freeze a high-risk wallet
await kycSDK.freezeWallet(suspiciousWallet);

// Later, unfreeze after review
await kycSDK.unfreezeWallet(reviewedWallet);
```

Emergency Controls

```
// Activate circuit breaker in case of security incident
await kycSDK.activateCircuitBreaker();

// All transfers will be blocked until deactivated

// After resolving the issue
await kycSDK.deactivateCircuitBreaker();
```

Using ZK Proofs for Privacy

```
import { ZkProofService } from '@gitdigital/solana-kyc-sdk';

// Generate a ZK proof (client-side)
const proof = await ZkProofService.generateProof({
  privateData: {
    age: 25,
    country: 'US',
  },
  signal: transactionHash,
});

// Verify the proof on-chain (without revealing data)
const isValid = await kycSDK.verifyProof(proof);
```

Testing

Running Integration Tests

```
# Start a local test validator
solana-test-validator --reset

# Run integration tests
npm run test:integration
```

Running E2E Tests

```
# Deploy programs to localnet
npm run deploy:localnet

# Run E2E tests
npm run test:e2e
```

Writing Custom Tests

```
import { expect } from 'chai';
import { kycSDK, KycLevel } from './kyc';

describe('KYC Integration', () => {
  it('should register and verify a user', async () => {
    const wallet = Keypair.generate().publicKey;

    await kycSDK.registerUser({
      wallet,
      kycLevel: KycLevel.Standard,
      countryCode: 'US',
      expiryTimestamp: Math.floor(Date.now() / 1000) + 365 * 24 * 60 *
60,
    });

    const status = await kycSDK.checkComplianceStatus(wallet);
    expect(status.status).to.equal('verified');
  });
});
```

Deployment

1. Build the Programs

```
anchor build
```

2. Deploy to Target Cluster

```
# Development (Devnet)
npm run deploy -- --cluster devnet

# Production (Mainnet)
npm run deploy -- --cluster mainnet
```

3. Verify Deployment

```
const info = await kycSDK.getRegistryInfo();
console.log('Registry deployed:', info);
```

Best Practices

1. **Always validate KYC before transfers** - Don't assume a user is verified
2. **Handle errors gracefully** - Provide meaningful error messages
3. **Keep KYC levels updated** - Monitor expiration dates
4. **Use circuit breaker for emergencies** - Have a plan for security incidents
5. **Test thoroughly** - Cover edge cases in your tests
6. **Follow AML regulations** - Stay compliant with local laws

---

Support

- **Documentation**: https://docs.gitdigital.products
- **GitHub Issues**: https://github.com/gitdigital-products/solana-kyc-compliance-sdk/issues
- **Discord**: https://discord.gg/gitdigital
- **Email**: support@gitdigital.products